Soft Computing – Project.

Project 2: Hopfield Network - Noisy Digit Recall

```
[1]
✓ 3s
import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_digits

rng = np.random.default_rng(0)
digits = load_digits()
chosen = [0, 1, 2, 7]                          # store karne wale digits

# 1. Patterns (average digit image -> bipolar +1/-1)
patterns = np.array([
    np.where(digits.images[digits.target == d].mean(axis=0).flatten() >= 7, 1, -1)
    for d in chosen])
P, N = patterns.shape

def show(imgs, titles, cols=3):
    rows = int(np.ceil(len(imgs) / cols)); plt.figure(figsize=(2.2 * cols, 2.4 * rows))
    for i, (im, t) in enumerate(zip(imgs, titles)):
        plt.subplot(rows, cols, i + 1); plt.imshow(im.reshape(8, 8), cmap="gray_r")
        plt.title(t, fontsize=9); plt.axis("off")
    plt.tight_layout(); plt.show()

show(patterns, [f"Stored: {d}" for d in chosen], cols=4)

# 2. Training rules
def train_hebb(p):
    W = p.T @ p / p.shape[1]; np.fill_diagonal(W, 0); return W

def train_pinv(p):
    X = p.T.astype(float); return X @ np.linalg.pinv(X)

W_hebb, W = train_hebb(patterns), train_pinv(patterns)
```

```
[1]
✓ 3s
# 3. Recall (asynchronous update) + energy
def energy(W, s): return -0.5 * s @ W @ s

def recall(W, s, max_sweeps=30):
    s = s.copy(); hist = [energy(W, s)]
    for _ in range(max_sweeps):
        changed = False
        for i in rng.permutation(len(s)):
            new = 1 if W[i] @ s >= 0 else -1
            if new != s[i]: s[i] = new; changed = True
        hist.append(energy(W, s))
        if not changed: break
    return s, hist

def add_noise(p, frac):
    s = p.copy(); s[rng.choice(len(p), int(frac * len(p)), replace=False)] *= -1; return s

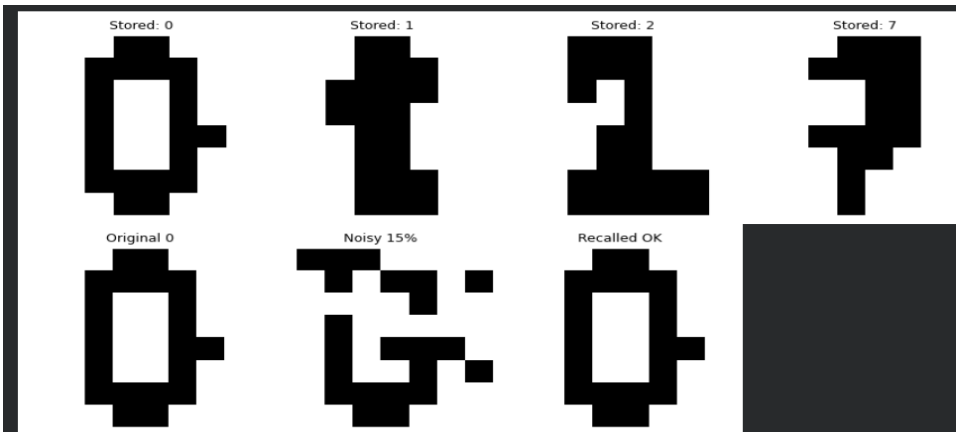
# 4. Demo
noise = 0.15; imgs, titles = [], []
for p, d in zip(patterns, chosen):
    noisy = add_noise(p, noise); rec, _ = recall(W, noisy)
    imgs += [p, noisy, rec]
    titles += [f"Original {d}", f"Noisy {int(noise*100)}%",
               "Recalled OK" if np.array_equal(rec, p) else "Recalled FAIL"]
show(imgs, titles)

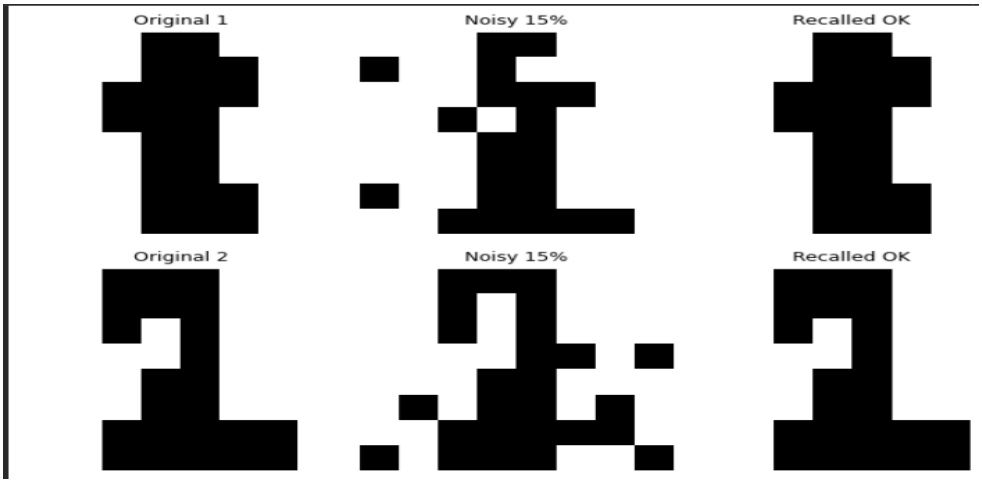
_, hist = recall(W, add_noise(patterns[0], 0.2))
plt.plot(hist, marker="o"); plt.xlabel("Sweep"); plt.ylabel("Energy"); plt.title("Energy decreases"); plt.show()
```

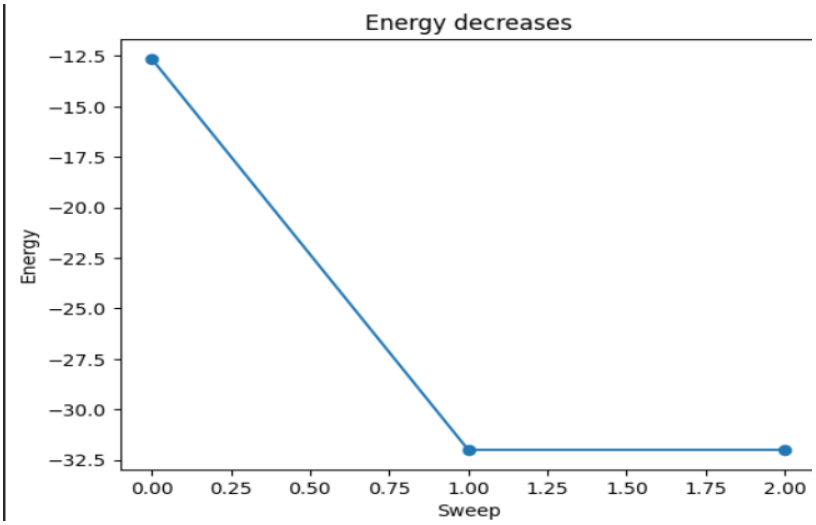
```python
# 5. Experiment A: noise vs recall (Hebbian vs Pseudo-inverse)
levels = np.arange(0, 0.55, 0.05); trials = 50
for name, Wx in [("Hebbian", W_hebb), ("Pseudo-inverse", W)]:
    acc = []
    for nl in levels:
        ok = 0
        for _ in range(trials):
            k = rng.integers(P); rec, _ = recall(Wx, add_noise(patterns[k], nl))
            ok += np.array_equal(rec, patterns[k])
        acc.append(ok / trials)
    plt.plot(levels * 100, acc, marker="o", label=name)
plt.legend(); plt.xlabel("Noise %"); plt.ylabel("Perfect recall rate")
plt.title("Robustness to noise"); plt.grid(alpha=.3); plt.show()

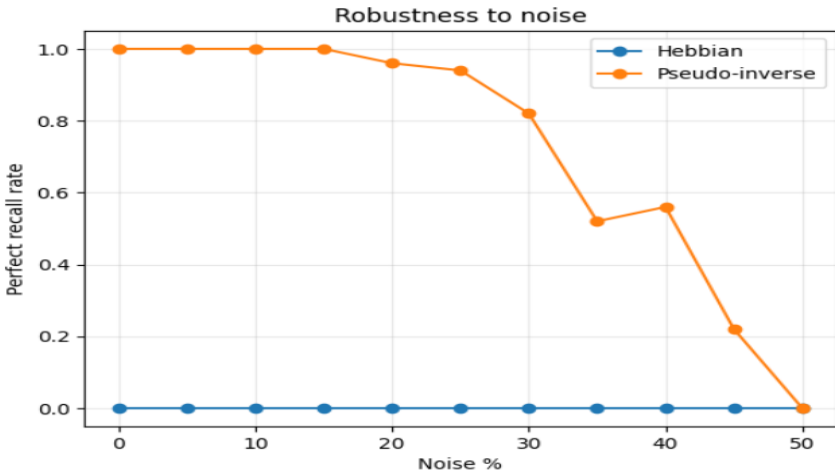
# 6. Experiment B: memory capacity
cap = []
for n in range(1, 11):
    pats = np.array([np.where(digits.images[digits.target == d].mean(0).flatten() >= 7, 1, -1)
                     for d in range(n)])
    Wn = train_pinv(pats)
    ok = sum(np.array_equal(recall(Wn, add_noise(pats[k], 0.1))[0], pats[k]) for k in range(n))
    cap.append(ok / n)
plt.plot(range(1, 11), cap, marker="s"); plt.xlabel("Stored patterns"); plt.ylabel("Recall rate (10% noise)")
plt.title(f"Memory capacity (N={N} neurons)"); plt.grid(alpha=.3); plt.show()
```

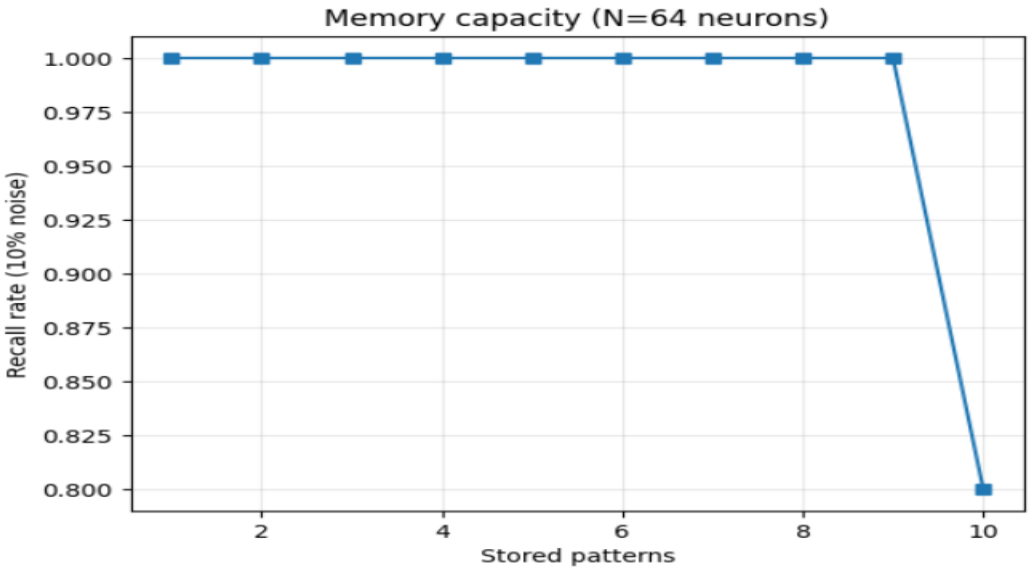
OUTPUT:






Energy decreases
Energy
Sweep
−12.5
−15.0
−17.5
−20.0
−22.5
−25.0
−27.5
−30.0
−32.5
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00


Robustness to noise
Hebbian
Pseudo-inverse
Perfect recall rate
Noise %
1.0
0.8
0.6
0.4
0.2
0.0
0
10
20
30
40
50


Memory capacity (N=64 neurons)
Recall rate (10% noise)
Stored patterns
1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.800
2
4
6
8
10